FULL-STACK WEB APPLICATION

Employee Management System

Angular | Spring Boot | MySQL

PROJECT REPORT

Student Name	Suhas G T
Student ID / Register Number	1BM25CD110
Course / Lab / Batch	[Enter exact course / lab / batch]
Submission Date	24-10-2026
GitHub Repository	[Paste repository URL after creating and pushing it]

Editable template - complete all bracketed fields, test outcomes, screenshots, and repository link before submission.

1. Student and Project Details

Field	Details
Student Name	Suhas G T
Student ID / Register Number	1BM25CD110
Project Title	Employee Management System
Course / Lab / Batch	[Enter exact course / lab / batch]
Submission Date	24-10-2026
GitHub Repository	[Paste URL after pushing the project]

2. Problem Statement and Proposed Solution

Problem statement

Maintaining employee information in paper registers or disconnected spreadsheets can make records difficult to update, locate, and keep consistent. Manual processes can also make it harder to confirm whether a record has been added, changed, or removed successfully.

Proposed solution

The Employee Management System provides one web interface for maintaining employee records. An Angular single-page frontend sends HTTP requests to a Spring Boot REST API. Spring Data JPA persists records in MySQL. The application supports creating, listing, updating, and deleting employees.

Objective, users, and scope

The objective is to demonstrate a complete frontend-to-backend-to-database CRUD flow for an employee entity with id, name, department, and salary. The target user is a small administrative user or lab evaluator who needs to manage employee records. This is a local lab application; authentication, role-based access, payroll, leave management, and cloud deployment are outside the current scope.

Expected result: valid employees can be created, listed, edited, and deleted; saved records remain after refreshing the browser. Mark the result as verified only after completing the tests in Section 9.

3. Webpage Design and Frontend Implementation

The responsive single-page interface has a top header, an employee details form, and a table of saved employees. The form contains Employee Name, Department, Salary, and Create Employee / Save Changes actions. The table contains ID, Name, Department, Salary, and Edit / Delete actions. Edit loads a selected row into the form; Delete asks for confirmation before making a request.

Technology	Role in this project
HTML template	Defines headings, form fields, validation feedback, the employee table, and buttons in employee.component.html.
CSS	Controls spacing, typography, colours, cards, table, status messages, and responsive behaviour.
TypeScript	Implements client-side form handling, events, validation flow, loading state, and success/error messages.
Angular 21	Provides standalone components, template binding, lifecycle handling, and the service-based UI structure.
Bootstrap 5	Provides responsive grid utilities, form controls, tables, buttons, alerts, and spinner styles; custom CSS adds the page design.
Spring Boot	Hosts the REST API and separates controller, service, and repository responsibilities.
Spring Data JPA / Hibernate	Maps Employee to the MySQL table and handles persistence.
MySQL	Stores employee records persistently.

4. Entities, Tables, and CRUD Operations

This version contains one entity because the requested scope is employee CRUD. A second artificial entity is not added because no second resource or relationship is required.

Entity / Table	Fields	CRUD behaviour
Employee / employees	id: BIGINT primary key, auto-increment; name: VARCHAR(100), required; department: VARCHAR(100), required; salary: DOUBLE, required and non-negative.	Create: POST saves a row. Read: GET lists or returns one. Update: PUT changes an existing row. Delete: DELETE removes the row.

Relationships and schema management

No foreign keys or entity relationships are required in this one-entity version. Hibernate uses ddl-auto=update for local development. A production application should use controlled database migrations.

CRUD data flow

- Create: form submit → Angular service POST → controller validates JSON → service/repository saves a row in MySQL → response is displayed.
- Read: Angular service GET → controller/service/repository read MySQL → JSON response fills the table.
- Update: Edit loads a row → Save changes sends PUT with its ID → backend updates the existing entity → table refreshes.
- Delete: Delete confirmation → Angular service DELETE → backend removes the row → refreshed list no longer contains it.

5. Frontend, Backend, Database, and Connectivity

Layer	Implementation
Frontend	Angular 21 standalone component; development URL http://localhost:4200.
Connectivity / API	Angular HttpClient sends JSON over HTTP to http://localhost:8080/api/employees.
Backend	Spring Boot REST controller, CRUD service, entity, repository, validation, and basic API errors at port 8080.
Database	MySQL schema employee_db and table employees, normally at localhost:3306.
Development tools	Use only tools actually used: editor, terminal, Java 21, Maven, Node.js/npm, MySQL Workbench, browser developer tools, and Git/GitHub.

REST endpoints

Method	Endpoint	Purpose / expected status
GET	/api/employees	List all employees - 200 OK
GET	/api/employees/{id}	Read one - 200 OK or 404
POST	/api/employees	Create employee - 201 Created
PUT	/api/employees/{id}	Update employee - 200 OK or 404
DELETE	/api/employees/{id}	Delete employee - 204 No Content or 404

6. Mock Data / JSON Server

A dummy JSON Server is not used. The application sends requests to a Spring Boot REST API and stores records in MySQL. Do not describe this project as using React or JSON Server.

7. Screenshot Evidence

Replace each placeholder with a screenshot from your actual running project. Use the file names in docs/SCREENSHOT_GUIDE.md. The placeholder areas are not evidence and must not remain in the submitted report.

Screenshot evidence - 1 and 2

SCREENSHOT 1

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 1 - Project setup

Show VS Code with backend packages and Angular source files. This demonstrates the project structure.

SCREENSHOT 2

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 2 - Home page / main UI

Show the employee form and employee table area in the browser. This demonstrates the main screen.

Screenshot evidence - 3 and 4

SCREENSHOT 3

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 3 - Create operation

Show a valid employee after submitting the form, with its row visible. This demonstrates Create.

SCREENSHOT 4

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 4 - Read / list operation

Show records returned from the backend in the table. This demonstrates Read/List.

Screenshot evidence - 5 and 6

SCREENSHOT 5

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 5 - Update operation

Show a changed value after editing a record; the ID should stay the same. This demonstrates Update.

SCREENSHOT 6

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 6 - Delete operation

Show the list after confirming deletion; the chosen row should be absent. This demonstrates Delete.

Screenshot evidence - 7 and 8

SCREENSHOT 7

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 7 - API / backend evidence

Show the GET response from /api/employees or a successful Spring Boot startup terminal. This demonstrates API/backend evidence.

SCREENSHOT 8

Replace this box with your actual screenshot

Capture from the running application - this is only a placeholder, not evidence.

Screenshot 8 - Database evidence

Show MySQL Workbench with the employees table and saved rows. This demonstrates persistence.

8. Source Code and GitHub

The source code is organised into backend/ and frontend/. Setup instructions, prerequisites, API endpoints, and Git commands are in the root README.md.

GitHub URL: [Paste URL after creating and pushing the repository]

Submission option: [GitHub repository / ZIP file]

For a ZIP submission, include source code, pom.xml, package.json, README, documentation, and real screenshots. Exclude generated folders such as node_modules, dist, target, and IDE caches. Never publish real passwords, tokens, or private data.

9. Testing and Results

Complete the Actual Result and Status columns only after running each case locally. Do not mark a test as passed based on expected behaviour alone.

Test	Expected result	Actual result / status
Create	Valid employee is saved and appears in the list.	[Fill after testing]
Read / refresh	Saved row remains after browser refresh.	[Fill after testing]
Update	Edited value appears, with the same ID.	[Fill after testing]
Delete	Confirmed record disappears from the list.	[Fill after testing]
Required fields	Empty required field blocks submit.	[Fill after testing]
Negative salary	Negative salary is rejected.	[Fill after testing]
API not found	Unknown ID returns 404 and an error message.	[Fill after testing]
Database persistence	MySQL rows match the latest UI state.	[Fill after testing]

10. Conclusion and Future Improvements

The project demonstrates a CRUD workflow using Angular, Spring Boot, Spring Data JPA, and MySQL. The frontend uses forms and a table to manage records, the backend exposes REST endpoints and validates input, and MySQL stores data between browser sessions. Possible improvements include login and roles, employee search and pagination, automated tests, schema migrations, audit logging, and deployment configuration.

Final submission checklist

[ ] Course / lab / batch details are completed accurately.

[ ] Both backend and frontend run locally.

[ ] All CRUD operations and validation have been tested.

[ ] Actual test outcomes are entered in Section 9.

[ ] All eight actual screenshots replace the placeholders in Section 7.

[ ] The GitHub URL works for the evaluator, or the requested ZIP is attached.

[ ] README contains correct setup and run instructions.

[ ] No real passwords, tokens, or sensitive data are included.